

Python Data Structures 2: Dictionaries And Sets



Suman

31 Jan, 2026 At 8:00 PM

Notes

Trimester-I

Recommended

Resource



Associated Lectures (1)



Description



Discussions

2.2 Session Dictionaries and Sets

In-Class Notes

The Concept

A dictionary stores **key-value pairs**. Think of it as a **real dictionary** - you look up a word (key) to find its definition (value).

Creating Dictionaries

```
# Empty dictionary
empty = {}

# Dictionary with values
person = {
    "name": "Alice",
    "age": 28,
    "city": "Mumbai"
}

# Using dict() constructor
scores = dict(math=95, science=87, english=92)
```

CRUD Operations (Create, Read, Update, Delete)

```
student = {"name": "Bob", "age": 20}

# CREATE - Add new key-value pair
student["grade"] = "A"
print(student) # {'name': 'Bob', 'age': 20, 'grade': 'A'}

# READ - Access value by key
print(student["name"]) # Bob

# UPDATE - Modify existing value
```

```
student["age"] = 21
print(student["age"]) # 21

# DELETE - Remove key-value pair
del student["grade"]
print(student) # {'name': 'Bob', 'age': 21}
```

The get() Method (Safe Access)

```
person = {"name": "Alice", "age": 28}

# Direct access - raises KeyError if key missing
# print(person["email"]) # KeyError: 'email'

# get() - returns None if key missing
print(person.get("email")) # None

# get() with default value
print(person.get("email", "Not provided")) # Not provided
```

Iterating Over Dictionaries

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# Iterate over keys
for key in student:
    print(key) # name, age, grade

# Iterate over values
for value in student.values():
    print(value) # Alice, 20, A

# Iterate over both (most useful)
for key, value in student.items():
    print(f"{key}: {value}")

# Output:
# name: Alice
# age: 20
# grade: A
```

Dictionary Methods

Method	Description	Example
keys()	Get all keys	student.keys()
values()	Get all values	student.values()
items()	Get key-value pairs	student.items()
get(k, d)	Get value or default	student.get("gpa", 0.0)
pop(k)	Remove & return value	student.pop("age")

Method	Description	Example
<code>update(d)</code>	Merge dictionaries	<code>student.update({"gpa": 3.8})</code>
<code>k in dict</code>	Check if key exists	<code>"name" in student</code>

Practical Example: Product Inventory

```
inventory = {
    "laptop": {"price": 999, "stock": 15},
    "mouse": {"price": 29, "stock": 50},
    "keyboard": {"price": 79, "stock": 30}
}

# Access nested dictionary
print(inventory["laptop"]["price"]) # 999

# Update stock after sale
inventory["mouse"]["stock"] -= 1

# Check if product exists
product = "monitor"
if product in inventory:
    print(f"Price: ${inventory[product]['price']}")
else:
    print(f"{product} not in inventory")
```

Section I: Sets & Data Structure Selection

The Concept

A set is an **unordered collection of unique elements**. Think of it as a **bag of unique tokens** - no duplicates allowed, and you cannot pick by position.

Creating Sets

```
# Using curly braces
fruits = {"apple", "banana", "cherry"}

# From list (removes duplicates)
numbers = [1, 2, 2, 3, 3, 3]
unique = set(numbers)
print(unique) # {1, 2, 3}

# Empty set (NOT {} - that's an empty dict!)
empty = set()
```

Set Operations

```

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Union (items in either set)
print(a | b)          # {1, 2, 3, 4, 5, 6}
print(a.union(b))     # Same result

# Intersection (items in both sets)
print(a & b)          # {3, 4}
print(a.intersection(b)) # Same result

# Difference (items in a but not b)
print(a - b)          # {1, 2}
print(a.difference(b)) # Same result

# Symmetric difference (items in one but not both)
print(a ^ b)          # {1, 2, 5, 6}
print(a.symmetric_difference(b)) # Same result

```

Set Methods

```

colors = {"red", "green", "blue"}

# Add element
colors.add("yellow")

# Remove element (raises error if not found)
colors.remove("red")

# Discard element (no error if not found)
colors.discard("purple") # No error

# Check membership (very fast!)
print("green" in colors) # True

```

Data Structure Selection Guide

Need	Best Choice	Why
Ordered collection, may change	List	Supports indexing, mutable
Fixed record of values	Tuple	Immutable, can be dict key
Key-value lookups	Dictionary	O(1) access by key
Unique items, no order needed	Set	Auto-removes duplicates, fast membership
Stack (LIFO)	List	<code>append()</code> and <code>pop()</code>
Queue (FIFO)	<code>collections.deque</code>	Efficient both ends

Practical Example: Removing Duplicates

```
# Problem: Find unique visitors
visitors = ["alice", "bob", "alice", "charlie", "bob", "diana"]

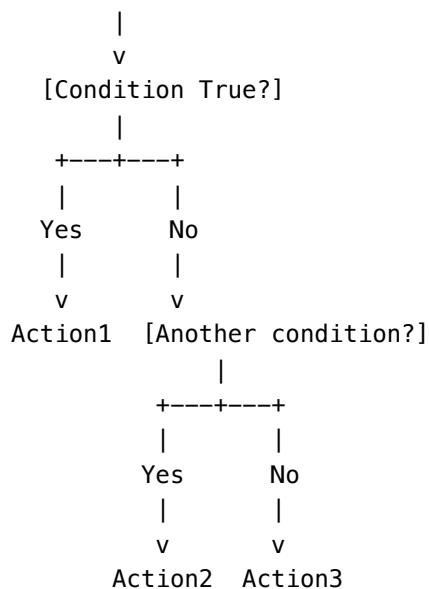
# Solution: Convert to set
unique_visitors = set(visitors)
print(f"Unique visitors: {unique_visitors}")
# Output: Unique visitors: {'alice', 'bob', 'charlie', 'diana'}

print(f"Total unique: {len(unique_visitors)}") # 4
```

Key Frameworks

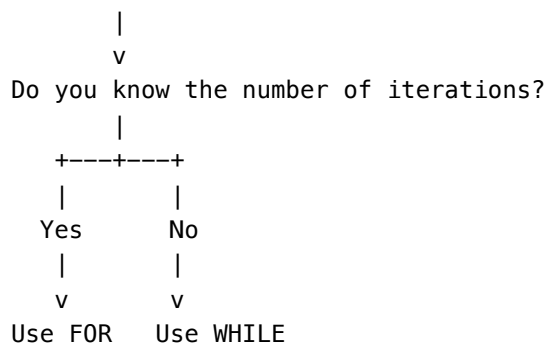
Decision Tree Pattern

Start: What condition to check?



Loop Selection Guide

Need to iterate?



Use for when:

- Iterating over a sequence (list, string, range)
- You know how many times to loop
- Processing each item in a collection

Error	Cause	Fix
<code>IndexError: list index out of range</code>	Accessing index that does not exist	Check <code>len()</code> before accessing
<code>KeyError</code>	Dictionary key does not exist	Use <code>get()</code> or check <code>in</code> first
<code>TypeError: 'tuple' object does not support item assignment</code>	Trying to modify tuple	Use list instead, or create new tuple
<code>TypeError: unhashable type: 'list'</code>	Using list as dict key or set element	Use tuple instead
<code>NameError</code>	Variable used before assignment	Initialize variable before loop
Infinite loop	While condition never becomes False	Ensure loop body changes condition

Key Takeaways

Conditionals direct program flow - `if` checks once, `elif` provides alternatives, `else` is the default

Comparison operators (`==` , `!=` , `<` , `>` , `<=` , `>=`) produce Boolean results; **logical operators** (`and` , `or` , `not`) combine them

for loops iterate a known number of times; **while loops** continue until a condition becomes False

break exits loops immediately; **continue** skips to the next iteration

Lists are ordered and mutable - use for collections that change

Tuples are ordered and immutable - use for fixed records and function returns

Dictionaries store key-value pairs - use for fast lookups by meaningful keys

Sets contain unique elements only - use for membership testing and removing duplicates

Next Session Preview

Session 2.2: Functions & Modular Code

You will learn to:

- Define reusable functions with `def`
- Work with parameters and return values
- Understand variable scope (local vs global)

- Write docstrings for documentation
- Apply functions to build modular programs

Preparation: Review the Contact Book mini-project - we will refactor it using functions!

Vishlesan i-Hub IIT Patna x Masai School